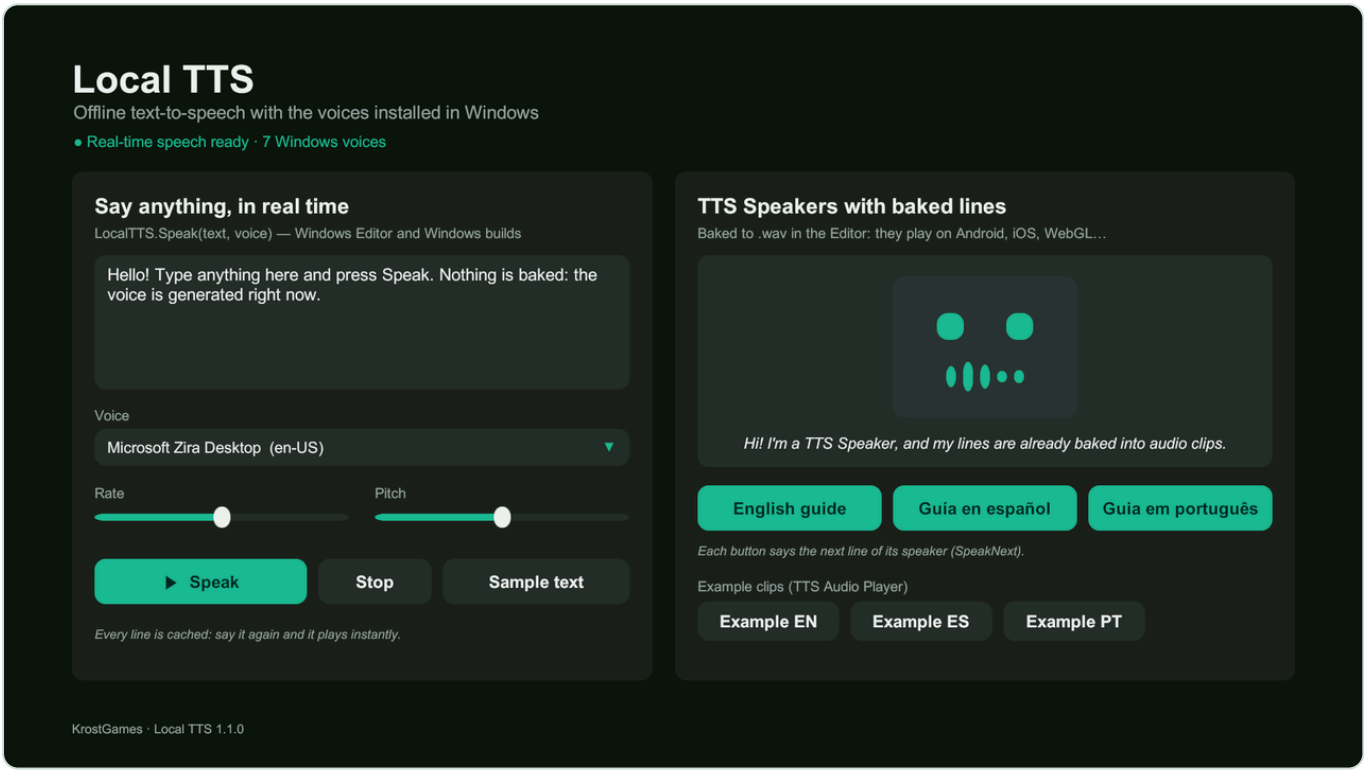
Local TTS

Offline text-to-speech for Unity with the voices installed in Windows

Speak in real time, or bake lines into .wav clips that play on every platform. No cloud, no API keys, no tokens.

Version 1.1.0 · Unity 2022.3 LTS or newer · KrostGames



Contents

1. What it does
2. Requirements
3. Installation
4. Quick start: three ways to use it
5. Real time or baked? What each platform does
6. The Generator window
7. TTS Speaker component
8. Line Library
9. Voice Profiles
10. Scripting
11. Batch files
12. Installing more voices
13. Folder structure
14. Upgrading from 1.0
15. FAQ and troubleshooting

What it does

- **Real-time speech, no bake:** `LocalTTS.Speak("Hello!")` synthesizes and plays a line on the spot, in the Windows Editor and in Windows builds. Repeated lines come from a memory cache and play instantly.
- **Bake once, play everywhere:** turn any line into a standard `.wav` AudioClip that plays on Android, iOS, WebGL, consoles, macOS or Linux.
- **TTS Speaker component:** write lines in the Inspector, listen with ▶, bake with one click, and call `Speak()` from UI buttons, XR interactions, Timeline or code. Speech goes through the GameObject's AudioSource, so it can be 3D and routed to a mixer.
- **Line Library:** lines your code speaks in Play Mode are recorded automatically, ready to bake so the same code also speaks on platforms without real-time synthesis.
- **Generator window:** type, preview from memory, save `.wav` files, batch-generate from `.txt` / `.csv`, drag clips from the history onto any AudioClip field.
- **Every Windows voice:** classic SAPI 5 "Desktop" voices and the Windows 10/11 OneCore voices (the ones you add in Settings > Speech), in any language you install.
- **Rate, pitch and volume** per line, voice profiles you can share between speakers, and SSML for fine control.
- **Editor UI in English, Spanish and Portuguese.**

Requirements

Unity	2022.3 LTS or newer (tested on Unity 6.3, 6.4 and 6.5)
Generating speech	Windows 10 or 11 (Editor or Windows build). Uses Windows PowerShell 5.1, which ships with Windows
Voices	At least one Windows voice installed (Windows includes one or more for its display language)
Playing baked clips	Any platform Unity supports

No internet connection is needed at any point.

Installation

1. Import the package (Assets > Import Package > Custom Package..., or from the Package Manager's *My Assets*). It installs into `Assets/KrostGames/LocalTTS`.
2. You can move the folder anywhere inside `Assets/`: the tools find their files on their own.
3. Open **Tools > Local TTS > Generator**. The status chip turns green ("Ready · N voices") when the speech engine has started.

Quick start: three ways to use it

1. Generator window (no code)

Tools > Local TTS > Generator. Type a line, pick a voice, press ▶ **Preview** (plays from memory, nothing is saved) and **Generate .wav** to save it as an asset. With *Assign to the selected object* on, the clip lands directly on the selected TTS Speaker, TTS Audio Player or Audio Source.

2. TTS Speaker component (Inspector)

GameObject > Audio > TTS Speaker (or *Add Component* > *KrostGames* > *Local TTS* > *TTS Speaker*). Write the lines, press ▶ on any line to hear it, and **Bake missing** when you want them in non-Windows builds. Wire `Speak()`, `SpeakNext()`, `SpeakLine(int)` or `Speak(string)` to any UnityEvent, such as a UI Button or an XR interactable.

3. One line of code

```
using KrostGames.TTS;

LocalTTS.Speak("Hello! This line was generated just now.");
```

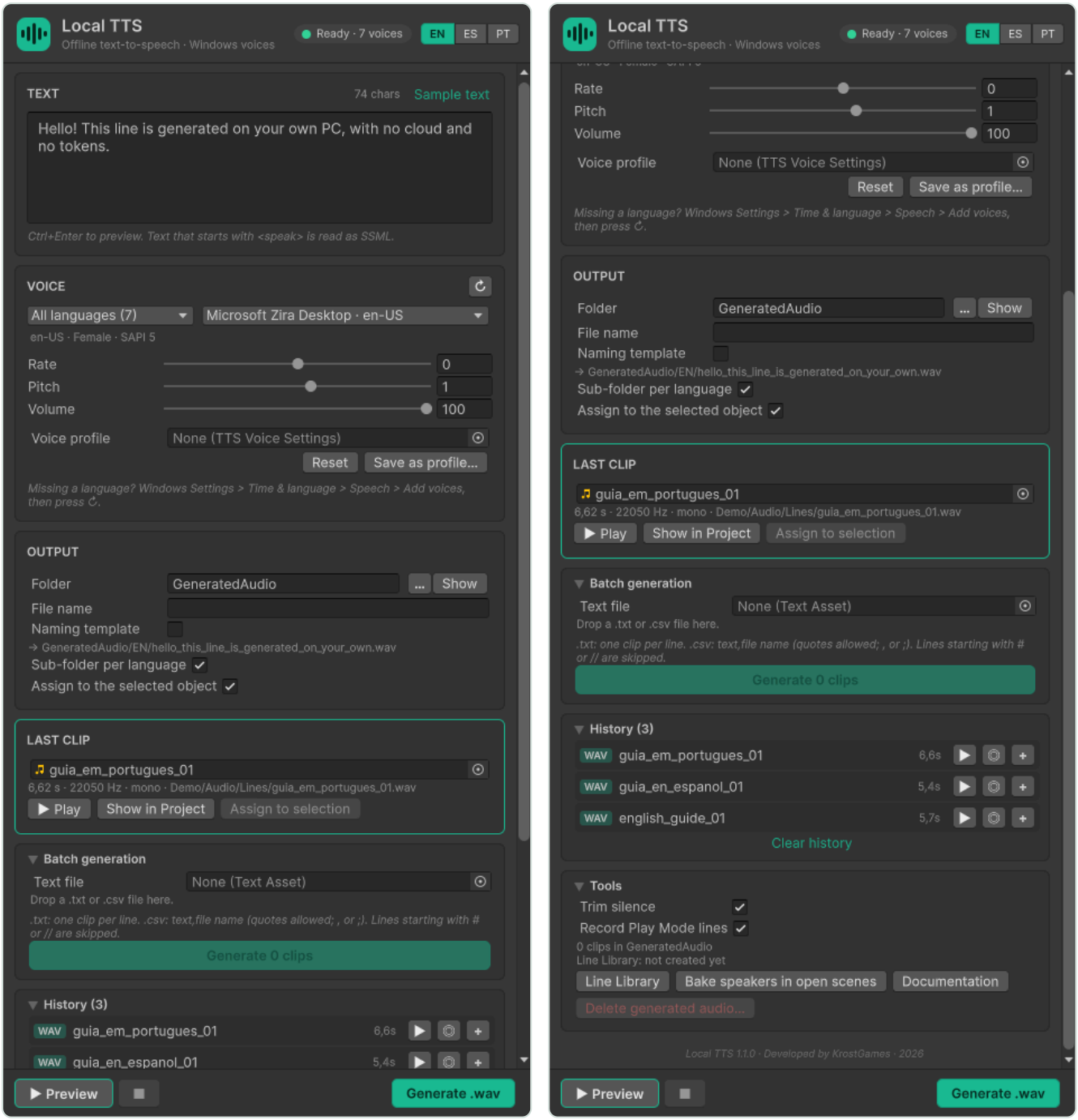
That's it: no component, no setup. Lines queue up on a shared 2D speaker.

Real time or baked? What each platform does

	Windows Editor	Windows build	Android, iOS, WebGL, consoles, macOS, Linux
Play baked clips	✅	✅	✅
Real-time speech (`LocalTTS.Speak` , speaker lines without a clip)	✅	✅	— plays the baked clip instead
Generate / bake `.wav` files	✅	—	—

A TTS Speaker in **Auto** mode does the right thing on each platform: it plays the baked clip when it matches the line's current text and voice, synthesizes in real time when it does not (Windows), and falls back to any baked clip elsewhere. Its Inspector tells you how many lines would be silent on the active build target.

The Generator window

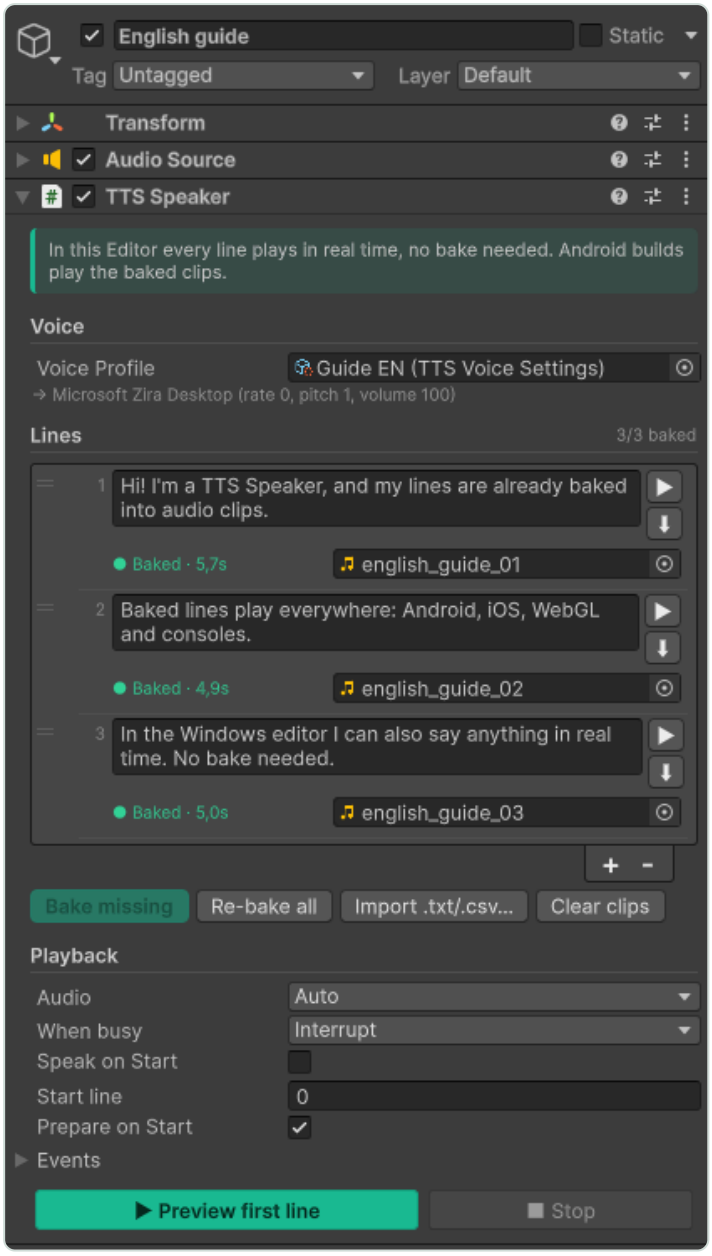


Tools > Local TTS > Generator: text, voice and output (left); last clip, batch, history and tools (right).

Area	What it does
Header	Engine status (click it to retry after an error) and the UI language (EN / ES / PT).
Text	The line to speak. `Ctrl+Enter` previews it. Text starting with `<speak>` is read as SSML.
Voice	Language filter, installed voices (SAPI and OneCore), rate, pitch and volume. Load a *Voice Profile* or save the current settings as one. ↻ reloads the voice list after installing voices.
Output	Output folder (default `GeneratedAudio/`), optional file name, naming template (`{lang}` `{id}` `{name}` `{voice}`), sub-folder per language, assign to selection.
Last clip	Play, show in Project, assign to the selection, or drag the card onto any AudioClip field.
Batch generation	Drop a `.txt` or `.csv` and generate every line. The Editor stays responsive; Cancel keeps the clips already finished.
History	The last 20 clips: play, ping, assign, or drag them out.
Tools	Trim silence, record Play Mode lines, Line Library, bake every speaker in the open scenes, documentation, move all generated audio to the trash.

File naming: with a file name, an existing file of that name is **replaced and keeps its references** (handy to re-record a line); without one, the name comes from the text and never overwrites anything.

TTS Speaker component

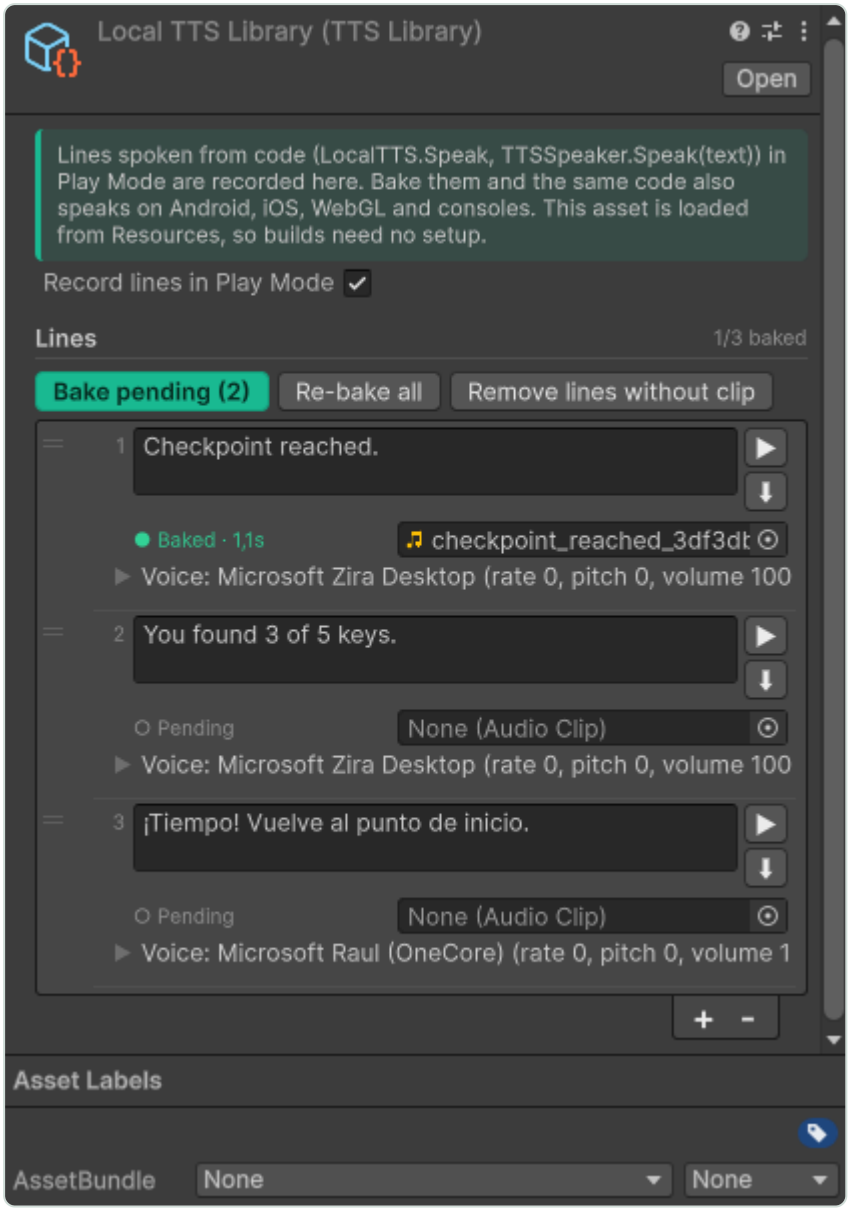


A TTS Speaker using a Voice Profile, with three baked lines.

Setting	
Voice Profile	Optional shared voice. When empty, the inline voice settings are used (installed-voice picker, language, rate, pitch, volume, ▶ test).
Lines	Text of each line, its baked clip and its status: ● Baked, ▲ Outdated (text or voice changed), ○ Real-time only. Reorder by dragging. **Bake missing**, **Re-bake all**, **Import .txt/.csv**, **Clear clips**.
Audio	*Auto* (recommended), *Realtime Only* (always synthesize, Windows), *Baked Only* (exactly what non-Windows builds do).
When busy	*Interrupt*, *Queue* or *Ignore* new lines while one is playing.
Speak on Start / Start line	Say a line when the scene starts.
Prepare on Start	Start the engine and synthesize the lines in the background, so the first line has no delay (Windows).
Events	`On Speech Started (string)` and `On Speech Finished (string)`. Finished is raised when a line plays to the end (not on `Stop()`).

Baked clips are named from the text and the voice settings, so baking the same line twice reuses the file and identical lines share one clip.

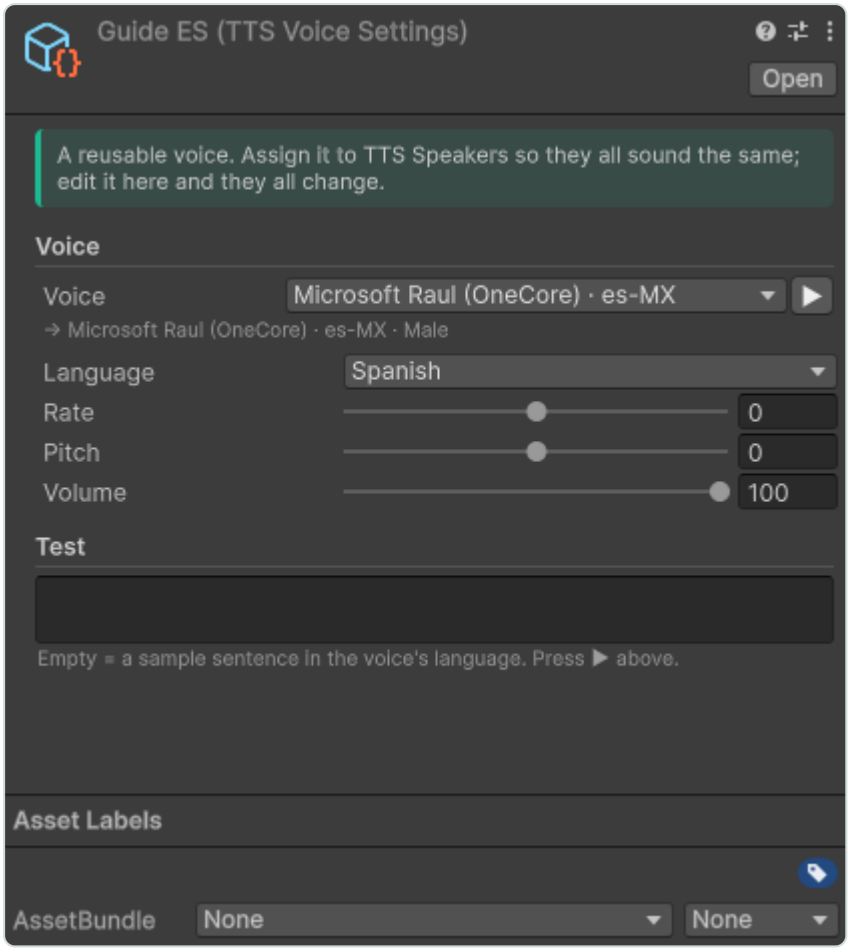
Line Library



The Line Library: lines recorded from code, one baked and two pending.

Lines spoken from code (`LocalTTS.Speak` , `TTSSpeaker.Speak(string)`) during Play Mode are added to the **Line Library** (`GeneratedAudio/Resources/LocalTTSLibrary.asset` , created on first use). Open it with **Tools > Local TTS > Line Library** and press **Bake pending**. From then on the same code finds those clips on every platform. Because the asset lives in a `Resources` folder, builds need no setup. When you build for a platform without real-time speech, a console warning tells you how many library lines are still unbaked. Recording can be turned off in the Generator's Tools or in the library itself.

Voice Profiles



A Voice Profile asset with the installed-voice picker and ▶ test.

Assets > Create > Local TTS > Voice Profile. A profile stores a voice, language, rate, pitch and volume. Assign it to several speakers ("Narrator", "Robot", "Guide") and edit it once to change all of them. The Inspector has the same voice picker and a test sentence.

Scripting

Everything is in the `KrostGames.TTS` namespace. Call it from Unity's main thread.

```
using KrostGames.TTS;
using UnityEngine;

public class Examples : MonoBehaviour
{
    public TTSSpeaker guide;
    public AudioSource sfx;
    public Transform mouth;

    async void Start()
    {
        // Choose a voice by full or partial name; language is the fallback if it is not installed.
        LocalTTS.DefaultVoice = new TTSVoiceOptions("Raul", TTSLanguage.Spanish, rate: 1, pitch: -1);
        LocalTTS.Prewarm();                             // start the engine early (loading screens)

        LocalTTS.Speak("¡Bienvenido!");                 // shared 2D speaker, lines queue up

        AudioClip clip = await LocalTTS.SynthesizeAsync("Score: 120");   // you own this clip
        sfx.PlayOneShot(clip);

        foreach (TTSVoice voice in await LocalTTS.GetVoicesAsync())
            Debug.Log(voice);                           // "Microsoft Zira Desktop (en-US)"...

        guide.OnSpeechFinished.AddListener(text => Debug.Log("Done: " + text));
        guide.SpeakLine(0);
    }

    void Update()
    {
        // Loudness (0..1) of the speech playing right now: drive a mouth, a light or a UI meter.
        mouth.localScale = new Vector3(1f, 0.2f + guide.Loudness, 1f);
    }
}
```

API	
`LocalTTS.Speak(text, voice = null)`	Speak through the shared speaker (baked clip from the Line Library if there is one, real time otherwise).
`LocalTTS.SynthesizeAsync(text, voice)`	New `AudioClip` with the line (Windows). `SynthesizeDataAsync` returns the raw PCM.
`LocalTTS.Prepare(text, voice)` / `Prewarm()`	Synthesize in the background / start the engine.
`LocalTTS.GetVoicesAsync()`, `Voices`, `ResolveVoice(options)`	Installed voices and which one some options select.
`LocalTTS.IsRealtimeSupported`, `State`, `StateChanged`, `LastError`	Engine status.
`LocalTTS.DefaultVoice`, `TrimSilence`, `CacheSize`, `StopAll()`, `Shutdown()`	Global settings and control.
`TTSSpeaker.Speak()`, `SpeakNext()`, `SpeakLine(i)`, `Speak(text)`, `Speak(text, voice)`, `Stop()`, `Pause()`, `Resume()`, `Prepare()`	Speaker control (all usable from UnityEvents).
`TTSSpeaker.IsSpeaking`, `IsPreparing`, `CurrentText`, `CurrentLine`, `Loudness`, `Lines`, `Voice`, `Mode`, `WhenBusy`	Speaker state and settings.

Editor scripting (`KrostGames.TTS.EditorTools`): `WindowsTTSEngine.GenerateAsync(text, voice, fileName, folder)`, `BakeAsync(text, voice)` and `BatchGenerateAsync(...)` write `.wav` assets. The 1.0 methods (`Generate`, `BatchGenerate`, `LivePreview`, `StopLivePreview`, `GetVoicesForLanguage`...) are still available.

SSML: text that starts with `<speak` is sent as SSML, e.g. `<speak version="1.0" xmlns="http://www.w3.org/2001/10/synthesis" xml:lang="en-US">Wait<break time="500ms"/> now!</speak>`.

Batch files

- **`.txt`:** one clip per line. Empty lines and lines starting with `#` or `//` are skipped. Commas inside a line are just text.
- **`.csv`:** `text,file name` per row; the file name is optional. Quote fields that contain commas (`"Hello, world",greeting`), write quotes as `""`. Semicolons work too (Excel in many locales). A first row `text,filename` is treated as a header.

Installing more voices

1. Windows **Settings > Time & language > Speech > Manage voices > Add voices** (on Windows 10: *Settings > Time & Language > Speech > Add voices*).
2. Pick the language (e.g. *Español (México)*, *Português (Brasil)*) and install it.
3. Press ↻ in the Generator. No restart needed.

Windows has two voice families and both are listed: **SAPI 5 "Desktop" voices** (e.g. *Microsoft Zira Desktop*) and **OneCore voices** (e.g. *Microsoft Raul*, shown with "(OneCore)"). Many languages add both.

Folder structure

```
Assets/KrostGames/LocalTTS/
├── README.md · LEEME.md · CHANGELOG.md · LICENSE.md · package.json
├── Documentation/    LocalTTS_Manual_EN.pdf · LocalTTS_Manual_ES.pdf
├── Runtime/          KrostGames.LocalTTS: LocalTTS, TTSSpeaker, TTSVoiceOptions, TTSVoiceSettings,
│                     TTSLibrary, TTSAudioPlayer, Engine/ (speech process, WAV)
├── Editor/           KrostGames.LocalTTS.Editor (Editor only): Generator, inspectors, bake tools
├── Demo/             LocalTTS_Demo.unity · Scripts/ · Voices/ · Audio/ (demo lines, examples)
└── GeneratedAudio/   Default output: EN/, ES/, PT/… and Resources/LocalTTSLibrary.asset
```

Project-wide settings are saved in `ProjectSettings/KrostGames.LocalTTS.asset` (shared with your team); the Generator's personal state in `UserSettings/`.

Upgrading from 1.0

- Namespaces are now `KrostGames.TTS` (runtime) and `KrostGames.TTS.EditorTools` (Editor). Replace `using KrostGames.LocalTTS.Runtime.Core;` with `using KrostGames.TTS;`.
- Assemblies are named `KrostGames.LocalTTS` and `KrostGames.LocalTTS.Editor`.
- Menus are now under **Tools > Local TTS**, and new installs go to `Assets/KrostGames/LocalTTS`.
- The example clips moved to `Demo/Audio/Examples/`.
- `TTSAudioPlayer`, `TTSVoiceSettings` and the `WindowsTTSEngine` 1.0 methods keep working. See CHANGELOG.md for the full list.

FAQ and troubleshooting

The status chip says "Windows only". Speech synthesis needs Windows. Clips baked on Windows play everywhere, so a macOS/Linux teammate can use them.

No voices for my language. Install a voice (see above) and press ↻.

Can I use real-time speech on Android / Meta Quest / iOS? Not real time: those platforms have no Windows voices. Bake the lines (TTS Speaker > *Bake missing*, or the Line Library) and they play there. The speaker Inspector warns about lines that would be silent on the active build target.

"Could not start Windows PowerShell" or the engine stays in Error. Real-time speech runs a hidden `powershell.exe` process. Some company policies block it; baked clips keep working. There is no need to change the PowerShell execution policy: the asset does not run script files.

My antivirus asked about PowerShell. That is the speech engine process mentioned above; it only uses the Windows speech APIs and closes with Unity or your game.

The voice sounds robotic. Quality depends on the installed Windows voice. Try the OneCore voices from *Add voices*, and adjust rate and pitch.

The first line takes a moment. Starting the engine takes about one or two seconds the first time. Call `LocalTTS.Prewarm()` or keep *Prepare on Start* on.

Does it need internet or cost anything per use? No. Everything runs on the local PC with Windows' own voices.

Local TTS 1.1.0 · Developed by **KrostGames** · 2026